Three Roads: Choosing the Right Portfolio Stack with AI

FlyRank AI Internship — General AI Fluency • Week 4

Student: Gungun Sharma

Date: 25 August 2026

Chosen stack: Next.js + Tailwind CSS

A constraint-driven stack decision for a professional portfolio

1. Executive Decision

I choose Road 2 — Next.js + Tailwind CSS, deployed on Vercel Hobby, with no backend/database for version 1. This is the best balance between my current frontend direction, a free-only constraint, professional presentation, maintainability and a realistic two-week delivery window.

2. My Four Constraints

1. Free only

The portfolio must be buildable and deployable with a ■0 budget. Paid hosting, a paid database, paid templates, and a paid custom domain are not required for the first version.

2. Honest skill level

I am strongest in HTML, CSS, JavaScript, Git/GitHub and frontend project work. I am currently building practical experience with React/Next.js and AI-enabled frontend workflows. My backend and database experience is more limited, so I should avoid adding backend infrastructure unless the portfolio genuinely needs it.

3. What the portfolio needs to do

The portfolio should present an introduction, about section, skills, internships/experience, selected projects, certifications/achievements, resume, contact information, and links to GitHub/LinkedIn. Projects should support screenshots, short explanations, technology tags, repository links and live-demo links where available.

4. How the work must be displayed

The work should be visual and recruiter-friendly: project cards, screenshots, concise case-study sections, technology badges, GitHub links, live demos and a downloadable/viewable resume. The first version does not need a database-driven CMS or user accounts. Content can remain static until there is a real need for dynamic updates.

3. Portfolio Sitemap & Content Map

Page / Area	What it must show
Home	Short introduction, role/goal, primary CTA, featured projects.
About	Short profile, learning direction, strengths and current focus.
Skills	Frontend, programming, AI/ML exposure, tools and workflow skills.
Experience	Internships and practical experience with concise responsibilities/outcomes.
Projects	Project cards with problem, solution, stack, screenshots, GitHub and live demo.
Case Studies	Optional deeper pages for 2–3 strongest projects; not every project needs a long page.
Certifications	Certificates/achievements displayed with title, issuer and evidence link where appropriate.
Resume	View/download resume.
Contact	Email/social links. A database-backed contact form is not required for v1.

Display Requirements

• Project title and one-line outcome

• Problem/context and what I built

• Technology stack

• Useful screenshots or visuals

• GitHub repository link

• Live demo link when available

• Short lessons/impact section

• Optional deeper case-study page for the strongest projects

Dynamic requirement for v1: none. The portfolio can remain static/local-content driven; a database-backed contact form is not required.

4. Three Genuine Stack Options

Road 1 — Simplest: HTML + CSS + JavaScript

How I would build: Create a responsive multi-section static portfolio using semantic HTML, modern CSS and vanilla JavaScript for small interactions such as navigation, filtering, theme toggle or form validation.

Free hosting: GitHub Pages is a natural free host for a static site and is available with GitHub Free for public repositories.

Backend: Not needed.

Advantages: Very easy to understand, low maintenance, fast, zero backend complexity, excellent for learning fundamentals.

Trade-off: More manual work as the site grows; reusable components, routing and larger UI systems are less convenient; advanced portfolio interactions require more custom JavaScript.

Fit: Good fit for a basic portfolio, but it leaves less room for the modern component-based workflow already relevant to my frontend internship.

Road 2 — Recommended: Next.js + Tailwind CSS

How I would build: Build a component-based portfolio in Next.js with Tailwind CSS. Keep project data/content local and render reusable project cards, experience sections and case-study pages. Add only lightweight client-side interactions where useful.

Free hosting: Vercel Hobby is the intended free deployment route for a personal/non-commercial portfolio. The site can also remain connected to a public GitHub repository.

Backend: Not needed for version 1. Next.js can remain a frontend/static-first solution; a server-side feature should only be added when there is a real requirement.

Advantages: Modern developer workflow, reusable components, strong project presentation, clean routing, easy Git-based deployment, scalable structure without forcing a database.

Trade-off: Higher learning curve than plain HTML/CSS/JS; requires understanding React/Next.js conventions and deployment configuration.

Fit: Best balance between my current frontend direction, the portfolio's actual needs, free-only constraint and a realistic two-week delivery window.

Road 3 — Most Powerful: Next.js + API/Serverless + Supabase

How I would build: Use Next.js for the frontend, server/API routes for dynamic behavior, and Supabase for database/auth/storage. Portfolio content could be managed dynamically and contact submissions could be stored.

Free hosting: Vercel for the application plus Supabase Free for backend services. Both have free offerings, but the free tiers have usage/feature limits and should be monitored.

Backend: Yes. This introduces API/serverless logic and a hosted database.

Advantages: Most flexible; supports dynamic content, analytics-style features, contact storage, authentication and future portfolio dashboards.

Trade-off: More moving parts, more maintenance, more failure points, more security considerations and unnecessary complexity for a portfolio whose main job is to showcase work.

Fit: Technically impressive but currently over-engineered for the actual requirement and my current backend experience.

5. Side-by-Side Comparison

Factor	Road 1 HTML/CSS/JS	Road 2 Next.js + Tailwind	Road 3 Next.js + API + Supabase
Difficulty	Low	Medium	High
Free-first fit	Excellent	Excellent	Good, but usage limits matter
Backend	No	No for v1	Yes
Maintenance	Low	Low–Medium	Medium–High
Portfolio presentation	Good	Excellent	Excellent
Reusability	Medium	High	High
Two-week feasibility	Very high	High	Medium
Matches current frontend direction	Good	Excellent	Good
Risk of over-engineering	Low	Low	High
Overall fit	8/10	9.5/10	7/10

6. Pressure-Test the Front-Runner

What breaks if I choose Road 1?

The site itself will work, but maintaining many repeated project sections can become tedious. As the portfolio grows, component reuse and routing become less convenient. It is the safest option technically, but not the strongest demonstration of the modern frontend skills I am actively developing.

What do I maintain if I choose Road 3?

I would maintain frontend code plus API/serverless logic plus a Supabase project. I would also need to think about environment variables, database structure, permissions, security rules and service limits. That is valuable for a full-stack product, but most of it does not improve the core portfolio experience.

Can I finish in two weeks?

Road 1: yes with the lowest risk. Road 2: yes if the scope stays focused on a polished portfolio rather than adding unnecessary features. Road 3: possible, but it creates avoidable integration and debugging risk.

Does it show my work the way it needs to be shown?

Road 2 does this particularly well because reusable project cards and case-study routes make it easy to show screenshots, tech stacks, GitHub repositories and live demos without introducing a database.

7. Final Decision & Rationale

I choose Road 2: Next.js + Tailwind CSS. It is not the most powerful option, and it is not the absolute simplest option. It is the smallest stack that still gives me a modern, reusable and professional portfolio structure. My portfolio is primarily a showcase, not a SaaS product, so I do not need authentication, a database or a custom backend in version 1. Next.js lets me build reusable components and project pages while keeping the first release simple. Tailwind CSS keeps styling consistent and makes it faster to iterate on responsive UI. I can deploy the portfolio on Vercel's free Hobby offering for a personal/non-commercial site and keep the source on GitHub. I can maintain this stack because it matches the frontend direction I am already practicing. If the portfolio later needs dynamic content, a contact database, a dashboard or other server-side features, I can add them deliberately instead of paying the complexity cost from day one.

Why I Did Not Choose the Other Two

Road 1: I did not choose it because, although it is the safest and easiest route, it gives me less leverage from reusable components and the modern frontend workflow I am trying to demonstrate.

Road 3: I did not choose it because a backend and database are not required by the portfolio's current content model. Adding them now would increase maintenance and security responsibility without improving the core deliverable enough to justify the complexity.

Can I Maintain This?

Yes. The architecture has a deliberate boundary: portfolio first, infrastructure second. I can maintain pages, components, project content and styling without a database or backend service.

Does It Show My Work Well?

Yes. The structure is optimized for project cards, screenshots, case studies, GitHub links and live demos, creating a fast path from who I am → what I can do → what I built → proof of work.

8. Two-Week Delivery Plan

Time	Goal
Days 1–2	Set up Next.js project, repository, basic layout, typography, navigation and responsive shell.
Days 3–5	Build Home, About and Skills sections; establish reusable UI components.
Days 6–9	Build project cards and 2–3 detailed case-study pages with screenshots, GitHub links and live demos.
Days 10–11	Add Experience, Certifications and Resume sections.
Day 12	Add Contact links and lightweight interactions; avoid backend unless a real need appears.
Day 13	Responsive testing, accessibility checks, broken-link checks, image optimization and content proofreading.
Day 14	Deploy, verify production build, update README and capture final portfolio/demo link.

9. Risks & Controls

Risk	**Control**
Too much time on animations	Keep motion subtle and prioritize content/UX.
Unnecessary backend features	No backend/database in v1 unless a real requirement appears.
Portfolio becomes too long	Feature 2–3 strongest projects; keep others concise.
Deployment problems	Push to GitHub early and deploy a minimal version before polishing.
Broken links	Run a final link and production-build check.
Heavy screenshots	Compress/optimize images and keep only useful visuals.
Next.js skill gap	Keep architecture simple and use reusable patterns rather than advanced features unnecessarily.

10. AI-Assisted Decision Method

1. State real constraints.
2. Generate three materially different stacks.
3. Compare cost, difficulty, backend need, maintenance and presentation quality.
4. Stress-test the simplest and most powerful routes.
5. Select the smallest stack that satisfies the actual portfolio needs.
6. Keep the reasoning explicit so the final decision remains mine.

11. Final One-Paragraph Rationale

I choose Next.js + Tailwind CSS because it gives me the best balance between simplicity, professional presentation and future flexibility. HTML/CSS/JavaScript would be easier, but it would make a growing portfolio more manual and would not demonstrate the component-based frontend workflow I am currently developing. A Next.js + API + Supabase stack would be more powerful, but it would add backend, database, security and maintenance responsibilities that my portfolio does not currently need. With Next.js and Tailwind, I can build a polished, responsive, reusable portfolio, keep the first version free, deploy it without a custom backend, and still have a clean path to add dynamic features later. Most importantly, I can maintain this stack and it shows my work well.

12. Sources Used for Free-Hosting Feasibility

GitHub Pages documentation: https://docs.github.com/en/pages/quickstart

GitHub Plans documentation: https://docs.github.com/en/get-started/learning-about-github/githubs-plans

Vercel Terms / Hobby plan reference: https://vercel.com/legal/terms

Supabase pricing (used only for the alternative-stack analysis): https://supabase.com/pricing

Note: Pricing and plan limits can change. The decision intentionally avoids making the portfolio dependent on paid services.